CSE 242 Assignment 3 - AI chat transcript

Ioannis Barous, 2252832

Used Claude (Claude Code) on Nov 10, 2026. Below are my prompts and Claude's answers in order (the commands it ran are not included).

Me (1):

hey, im doing assignment 3 for CSE 242 (machine learning) at ucsc. its on SVMs, k-means and knn.
everything is in ~/Downloads/CSE242_Assignment3: README.md has the whole assignment, then assignment3_template.py is q1, kmeans_template.py is q2, noisy_classification_template.py is q3, and requirements.txt
some rules for the class:

- we're allowed to use AI on the programming assignments but I have to submit the chat transcript
- don't change the templates structure. same function names, arguments, return values, constants, command line args. just fill in the TODOs
- follow the README exactly (seeds, probabilities, C and gamma, k values, 20 trials, output files etc)
- k-means in q2 has to be from scratch with numpy, no sklearn KMeans. q3 can use sklearn.neighbors

I need to turn in the 3 finished .py files, a pdf writeup (svm derivation + margin, kmeans plots + discussion, the accuracy comparison), the csv from q3, and the transcript
can you read the README and all 3 templates first and tell me every question, how many points, what it wants, which function its in and what needs to go in the writeup? dont solve anything yet

Claude:

I read everything in `~/Downloads/CSE242_Assignment3`. The folder also has `CLASS_README.md`, which only says to submit on Canvas, and `Assignment3_prompts.md`, which is your list of planned prompts (this is #1). The assignment is worth 100 points.

Q1: Hard-margin linear SVM (30 pts), `assignment3_template.py`

Six 2D points: IDs 1–3 are +1 and IDs 4–6 are −1.

Part	Pts	What it wants	Function
(a)	10	Plot the data (`--task plot` saves `svm_data.png`) and pick the support vectors by eye. Return their **1-based IDs**.	`identify_support_vectors()`
(b)	10	Derive the max-margin line `w1·x1 + w2·x2 + b = 0` from the support vectors. Hard margin, no slack, **keep b**.	`derive_svm_separator()`
(b)	10	Geometric margin = distance from the boundary to the closest points. If you also give the full width, label it separately.	same; returns `(weights, bias, geometric_margin)`

Writeup: the plot, which support vectors and why, the full derivation by hand, and the margin with your convention stated. The README says a library SVM fit alone doesn't count as the math.

Q2: K-means from scratch (30 pts), `kmeans_template.py`

Part	Pts	What it wants	Function
(a)	15	Assignment step: nearest centroid, returns an int array of shape `(n,)`, with a consistent tie rule.	`update_assignments(data, centroids)`
		Centroid step: returns a new `(k, d)` array without changing the input, with a documented empty-cluster rule. NumPy only, no library KMeans.	`update_centroids(data, centroids, assignments)`
(b)	15	Use `make_toy_data()` (4000 points, 4 blobs, std 0.60, random_state 0). Run k = 2, 3, 4, 6, 10 with a seeded RNG and `get_initial_clusters()`, then save one plot per k showing assignments and final centroids.	`run_experiments(output_dir, seed, max_iterations)`

- `kmeans()` and the initializer are provided. The initializer samples with replacement, so duplicate starting centroids can happen, and your empty-cluster rule has to handle that.
- The true labels can only go in an optional reference plot, never into the clustering.
- Plots go to `results/kmeans/`. Run it with `--seed 0 --max-iterations 100`.

Writeup: the tie rule and the empty-cluster rule, all 5 plots, and a short paragraph on what changes as k grows compared with the 4 real blobs.

Q3: Noisy-label SVM + K-NN repair (40 pts), `noisy_classification_template.py`

Data: separate train and test sets, each with 1000 positives ~ N((1,1), I) and 1000 negatives ~ N((–1,–1), 3I). `generate_clean_split()` is provided. Test labels stay clean.

Part	Pts	What it wants	Function
(a)	20	Flip +1 labels with prob **0.35** and –1 labels with prob **0.20**, based on the original labels so nothing flips twice. Return a new array.	`corrupt_training_labels(clean_labels, rng)`
		Baseline: RBF SVM with **C=1, gamma=0.01** (`train_and_evaluate()` is provided). Keep the data fixed and repeat corruption and training **20 times** with a seed. Report the mean clean-test accuracy.	baseline part of `run_comparison()`
(b)	20	For each training point, take its **5 nearest other points** (Euclidean) and use their majority noisy label. Compute every repair from the original noisy array, not one at a time. Use a consistent rule for distance ties. Only training features and noisy labels can go in. `sklearn.neighbors` is allowed.	`repair_labels_knn(train_features, noisy_labels, neighbors)`
		Each trial trains a fresh SVM on the repaired labels, using the same features and noisy labels as the baseline. Save a CSV with columns `trial, baseline_accuracy, knn_accuracy`.	comparison part of `run_comparison()`

- Default output is `results/noise_comparison.csv`. Run it with `--repeats 20 --seed 0`. `--neighbors` only accepts 5.

Writeup: the setup, both means and their difference (a per-trial table is optional), and whether K-NN repair helped and why it might help or hurt. The README says improvement isn't guaranteed, and no parameter sweep is needed.

Things to watch before prompt #2

- `requirements.txt` **doesn't include scikit-learn**, but `make_toy_data()` and `train_and_evaluate()` both need it. It only has `autograder-py==0.5.7`, numpy and matplotlib. Your prompt #2 already plans to install scikit-learn and pandas separately, which covers this.
- Solution modes raise `NotImplementedError` until the TODOs are filled. `--task plot` should already work.
- The README's deliverables also list "acknowledgment of outside help", so the writeup needs that section as well as the attached transcript.

Me (2):

ok can you set it up like the README says. make a .venv (python 3.9+), activate it, pip install requirements.txt. the code uses sklearn and I'll want pandas for the writeup so install scikit-learn and pandas too, they're not in requirements.txt. then run python assignment3_template.py --task plot and show me svm_data.png. tell me if anything breaks

Claude:

Setup worked and nothing broke. The plot is saved at `~/Downloads/CSE242_Assignment3/svm_data.png`.

Environment: I made `.venv` with Python 3.9.6, which meets the 3.9+ requirement.

Package	Version	From

autograder-py	0.5.7	requirements.txt
numpy	2.0.2	requirements.txt
matplotlib	3.9.4	requirements.txt
scikit-learn	1.6.1	added
pandas	2.3.3	added

pip also installed what those packages need, including scipy. The imports (`sklearn`, `pandas`, `numpy`, `matplotlib`) load without errors.

Plot: `python assignment3_template.py --task plot` ran cleanly. It shows the six numbered points: the +1 points (1, 2, 3) as blue circles and the −1 points (4, 5, 6) as orange squares.

Notes:

- Shell state doesn't carry over between my commands, so I activate the venv each time I run something. In your own terminal, activate it once with:

  `source ~/Downloads/CSE242_Assignment3/.venv/bin/activate`
- Because scikit-learn and pandas aren't in `requirements.txt`, a grader who installs only from that file won't have sklearn. It might be worth saying in the writeup that you installed them separately.

Me (3):

lets do q1 in assignment3_template.py

a) from the plot, which points are the support vectors? fill in identify_support_vectors() so it returns their ids (1-based like the table). and explain why those ones in normal words, I need it for the writeup

b) derive the max margin line w1x1 + w2x2 + b = 0 using just the support vectors. I want it done by hand step by step, set up y_i(w*x_i + b) = 1 for the support vectors and solve for w and b. keep b in there. then the geometric margin = distance from the line to the closest points, so 1/||w||. also put the full width 2/||w|| but label it separately so its not confusing. fill in derive_svm_separator() to return (weights, bias, geometric_margin)

after that check it works. every point should have y_i(w*x_i + b) >= 1, with = 1 only for the support vectors. you can also compare with sklearn SVC(kernel="linear", C=1e6) but only as a check, don't put that in the actual function.
run python assignment3_template.py --task svm and show me the output.
also make a plot with the line, both margin lines and the support vectors circled for the report. and write out the full derivation so I can paste it in

Claude:

Q1 is finished and runs cleanly. The support vectors are points 1, 5 and 6, and the line is 2x1 + 2x2 − 3 = 0. The geometric margin (line to closest points) is 0.3536. The full width between the two margin lines is 0.7071, labeled separately.

Output of `python assignment3_template.py --task svm`:

```
Support vector point IDs: [1, 5, 6]
Weights: (2.0, 2.0); bias: -3.0; geometric margin: 0.35355339059327373
```

Checks:

- **Constraints:** y_i(w·x_i + b) is exactly 1 for points 1, 5 and 6, and 3 for points 2, 3 and 4. So every point is ≥ 1, and only the support vectors are at = 1.
- **sklearn:** `SVC(kernel="linear", C=1e6)` gives the same w = (2, 2), b = −3 and support vectors {1, 5, 6}. I ran it in a separate script, so it isn't in the template.
- **Dual check:** the support-vector weights come out as $\alpha_1 = 4$, $\alpha_5 = \alpha_6 = 2$. They're all positive, which confirms this is the true maximum-margin line and not just a line that fits the three equations.

What changed: only the two TODOs in `assignment3_template.py`. I added a short comment with the reasoning and an `import math` inside the function. Names, arguments, return values and command-line options are unchanged.

Derivation to paste

(a) Support vectors: points 1 (1,1), 5 (1,0), 6 (0,1). The gap between the classes runs diagonally. Point 1 is the positive point closest to the negatives. Points 5 and 6 are the negative points closest to the positives, both on the line x1 + x2 = 1, while point 1 is on the parallel line x1 + x2 = 2. The widest strip between the classes lies between those two lines, so these three points hold up the margin. Points 2, 3 and 4 are farther away, and moving them a little wouldn't change the boundary.

(b) The support vectors satisfy $y_i(w \cdot x_i + b) = 1$:

- Point 1 (1,1), y=+1: w1 + w2 + b = 1 (i)
- Point 5 (1,0), y=–1: –(w1 + b) = 1 ⇒ w1 + b = –1 (ii)
- Point 6 (0,1), y=–1: –(w2 + b) = 1 ⇒ w2 + b = –1 (iii)

Solving: (i)–(ii) gives w2 = 2, (i)–(iii) gives w1 = 2, and (ii) gives b = –1 – 2 = –3.

Boundary: 2x1 + 2x2 – 3 = 0, which is the same as x1 + x2 = 1.5. The margin lines are x1 + x2 = 2 (through point 1) and x1 + x2 = 1 (through points 5 and 6).

w		= √8 = 2√2.

- **Geometric margin** (boundary to closest points) = 1/‖w‖ = 1/(2√2) = √2/4 ≈ **0.3536**
- **Full margin width** (between the two margin lines) = 2/‖w‖ = 1/√2 ≈ **0.7071**

Files

- `results/svm_boundary.png`: the report plot, with the boundary, both dashed margin lines and the support vectors circled.
- `q1_derivation.md`: the complete derivation, with the dual check and the table of all six points. Paste it from there; it has more than what's above.
- The check-and-plot script is outside the assignment folder, at `/private/tmp/claude-501/-Users-giannis--claude-spares-spare-W606gy/a4b466c8-9919-45a0-ad16-390f944e1ad1/scratchpad/q1_check_and_plot.py`.

Me (4):

now q2 in kmeans_template.py

a) write update_assignments() and update_centroids() from scratch with numpy.
if a point is tied between two centroids just give it the lower index (argmin already does that), and write that in the docstring. empty cluster = keep its old centroid, also write that in the docstring. dont change the centroids array in place

b) fill in run_experiments(). load make_toy_data(), make rng = np.random.default_rng(seed), and for each k in K_VALUES (2,3,4,6,10) get the starting centroids with get_initial_clusters() and run kmeans(). save one plot per k in output_dir, points colored by cluster and the final centroids as big X's. dont use the real blob labels for the clustering, but you can save one extra plot of the real labels to compare.

run python kmeans_template.py --seed 0 --max-iterations 100 and show me all the plots.
then write a short paragraph on what happens as k goes up and how it compares to the 4 real blobs (k less than 4 should merge blobs, k=4 should mostly match them, more than 4 splits real blobs up). if any run got stuck in a bad spot, like 2 centroids in one blob, mention it

Claude:

I've reviewed the k-means plots and checked them against the true blobs. Next I'm sending you the plots and writing the paragraph.

Q2 is done and `python kmeans_template.py --seed 0 --max-iterations 100` runs cleanly. It saves `kmeans_k2/3/4/6/10.png` and `true_labels_reference.png` in `results/kmeans/`.

(a) Update steps, NumPy only, written in the docstrings:

- `update_assignments()`**:** computes the squared distance from each point to each centroid and assigns the point to the closest one. **Tie rule:** a point equally close to two centroids goes to the lower index, which is what `np.argmin` does. This also handles duplicate starting centroids, since only the lower-index copy gets points.

- `update_centroids()`: copies the centroid array, then sets each nonempty cluster's centroid to the mean of its points. **Empty-cluster rule:** an empty cluster keeps its previous centroid. The input array is never modified.
- **Small hand-made test:** a point exactly between two centroids went to index 0. The empty clusters kept their old centroids, and the input array was unchanged afterwards.

(b) Output:

```
k= 2  inertia=  15222.21  cluster sizes=[1017, 2983]
k= 3  inertia=   6826.13  cluster sizes=[1008, 1992, 1000]
k= 4  inertia=   2794.24  cluster sizes=[1000, 998, 1002, 1000]
k= 6  inertia=   2396.79  cluster sizes=[1000, 299, 997, 407, 319, 978]
k=10  inertia=   1914.79  cluster sizes=[365, 350, 253, 261, 988, 997, 123, 213, 285, 165]
```

The clustering never uses the true labels; they only feed the reference plot. I used them in a separate check script, outside the assignment folder, to compare clusters against the blobs:

- Every run converged: one more update changed nothing.
- No run started with duplicate centroids, and none ended with an empty cluster.
- **Nothing got stuck badly.** No run with k ≤ 4 put two centroids in one blob while merging two others. The one oddity is that k=6 and k=10 split the blobs very unevenly, described below.

Paragraph for the writeup

As k increases, the total within-cluster squared distance (inertia) drops: 15222 at k=2, 6826 at k=3, 2794 at k=4, 2397 at k=6 and 1915 at k=10. The big drops stop at k=4, which matches the four groups that generated the data.

With **k < 4**, K-means has to merge real blobs. At k=2, the isolated top blob gets its own cluster and the other three bottom blobs become one cluster of 2,983 points, with its centroid in the empty space between them. At k=3, the bottom-right blob separates out, and the two middle blobs (around (1, 4.3) and (−1.5, 2.9)) are still merged.

At **k = 4**, the clusters match the true blobs almost exactly. The cluster sizes are 1000/998/1002/1000, only 9 of 4000 points are assigned to a different blob than their true one, and the centroids sit within about 0.02 of the true blob means.

With **k > 4**, K-means splits real blobs into pieces. The splits aren't even, because they depend on where the random starting points landed. At k=6, the left-middle blob is cut into three parts while the other three stay whole. At k=10, the bottom-right blob is cut into five parts and the top blob into three, while the two middle blobs each stay in one cluster. So inertia keeps falling as k grows, but beyond k=4 the extra clusters only split real groups without finding new ones. No run got stuck with two centroids sharing one blob while two other blobs were merged.

Files: the analysis script is at `/private/tmp/claude-501/-Users-giannis--claude-spares-spare-W606gy/a4b466c8-9919-45a0-ad16-390f944e1ad1/scratchpad/q2_check.py`. All the numbers in the paragraph come from the runs above.

Me (5):

q3 now, noisy_classification_template.py

a) corrupt_training_labels(): decide the flips from the original labels. +1 flips with prob 0.35, -1 flips with prob 0.20. one random draw per point so nothing gets flipped twice. return a new array, dont change clean_labels

b) repair_labels_knn(): use sklearn NearestNeighbors (euclidean) on the training features only. ask for n_neighbors+1 and drop the point itself so you get the 5 closest OTHER points. new label = majority vote of the neighbors noisy labels. do it all at once from the original noisy array, not one by one updating as you go. write down how distance ties are handled (sklearn order / lower index). it should never see the clean labels or test labels

then run_comparison(): rng = np.random.default_rng(seed). make the clean train set and a separate clean test set ONCE (1000 per class each). then for each of the 20 trials: corrupt the train labels, train the baseline with train_and_evaluate(), fix those same noisy labels with knn, train a new svm on the fixed labels, test both on the clean test set.
save a csv to output_path with exactly these columns: trial, baseline_accuracy, knn_accuracy. one row per trial, make the results folder if it doesn't exist. print the baseline mean, the knn mean and the difference

run python noisy_classification_template.py --repeats 20 --seed 0 and show me the output and the csv.
then a short explanation of whether knn helped or not and why. like it can help because neighbors outvote random flips where the classes dont overlap much, but it can hurt where they do overlap, and the noise isnt even (35% of positives flipped) so in some areas the majority could be wrong. plus the negative class is way more spread out (3I) so it overlaps the positives

Claude:

Q3 is finished and `python noisy_classification_template.py --repeats 20 --seed 0` runs in about 7 seconds. K-NN repair helped in all 20 trials: mean accuracy went from **0.8280 to 0.8644 (+0.0364)**. That's close to 0.8670, which is what the same SVM gets when trained on the clean labels.

(a) `corrupt_training_labels()`**:** each point's flip probability comes from its original label (0.35 for +1, 0.20 for −1). Each point gets one random draw, so nothing can flip twice. It returns a copy, and I checked that `clean_labels` is never changed. Over the 20 trials, about 0.339 of positives and 0.201 of negatives were flipped.

(b) `repair_labels_knn()`**:**

- It uses sklearn `NearestNeighbors` (Euclidean) on the training features only. It asks for 6 neighbors and drops the point itself, leaving the 5 closest other points.
- Each new label is the majority of those 5 neighbors' noisy labels. All of them are computed from the same original noisy array and written to a new one.
- **Distance ties:** neighbors are sorted by distance, then by row index, so with equal distances the lower index wins. This is in the docstring.
- I checked that shuffling the rows doesn't change the result. The function never sees the clean or test labels.

`run_comparison()`**:** one generator, `np.random.default_rng(seed)`. It makes the clean train set and a separate clean test set once. Each trial corrupts the labels, trains the baseline, repairs those same noisy labels, trains a fresh SVM on them, and scores both on the clean test set. It creates the folder, writes the CSV and prints the means.

Output (the CSV has the same numbers):

```
trial  1: baseline=0.8345  knn=0.8615
trial  2: baseline=0.8330  knn=0.8720
trial  3: baseline=0.8440  knn=0.8675
trial  4: baseline=0.8445  knn=0.8700
trial  5: baseline=0.8395  knn=0.8725
trial  6: baseline=0.8410  knn=0.8715
trial  7: baseline=0.8405  knn=0.8690
trial  8: baseline=0.7930  knn=0.8480
trial  9: baseline=0.8240  knn=0.8615
trial 10: baseline=0.8425  knn=0.8715
trial 11: baseline=0.8220  knn=0.8600
trial 12: baseline=0.8190  knn=0.8645
trial 13: baseline=0.8470  knn=0.8645
trial 14: baseline=0.8415  knn=0.8715
trial 15: baseline=0.8160  knn=0.8715
trial 16: baseline=0.8285  knn=0.8485
trial 17: baseline=0.8235  knn=0.8695
trial 18: baseline=0.8430  knn=0.8700
trial 19: baseline=0.7840  knn=0.8335
trial 20: baseline=0.7980  knn=0.8690
Mean baseline accuracy: 0.8280
Mean K-NN repair accuracy: 0.8644
Difference (K-NN - baseline): +0.0364
```

`results/noise_comparison.csv` has the header `trial,baseline_accuracy,knn_accuracy` and rows 1–20.

Explanation for the writeup

I also compared the training labels against the clean ones, in a separate analysis script only. The result changes the story a bit: **repair didn't make the training labels more correct overall.** About 27.0% of labels were wrong before repair and

27.3% after. On average each trial fixed 382 wrong labels and broke 387 correct ones. It still helped the SVM, for three reasons:

- **It helps where the classes are well separated.** A random flip there is usually outvoted by its 5 correctly labeled neighbors. In those regions the share of wrong labels fell from 0.269 to 0.228. The flips the baseline SVM sees are scattered at random, and the ones deep inside the other class pull its boundary off.
- **It hurts where the classes overlap.** There, the neighbors' labels are close to a coin flip, so the vote is often wrong. In the area where both classes are about equally likely, the share of wrong labels went from 0.279 to 0.509. The negative class has covariance 3I, so it's much more spread out and reaches into the positive region. Many genuine negatives there get relabeled +1, which counts as "breaking" a correct label.
- **The uneven noise pushes the vote one way.** Since 35% of positives are flipped but only 20% of negatives, the noisy labels are 43% positive instead of 50%. Near the boundary the majority leans negative. After repair only 41% of labels are positive, and 36.1% of true positives end up labeled wrong.
- **Why the SVM still gains:** repair makes labels locally consistent, so each area mostly carries the label of the class that is more common there. For comparison, I also labeled every training point with the best possible rule, the one computed from the two known Gaussians. Disagreement with that rule dropped from 0.331 to 0.233. The SVM learns that pattern better than random scattered flips, which is why test accuracy got close to training on clean labels. The gain isn't guaranteed, though: with more overlap or more lopsided noise, the errors in the overlap zone could cost more than the fixed flips save.

The analysis script is at `/private/tmp/claude-501/-Users-giannis--claude-spares-spare-W606gy/a4b466c8-9919-45a0-ad16-390f944e1ad1/scratchpad/q3_check.py`.

Me (6):

can you go through the README line by line and check everything against what we did? params, seeds, k values, 20 trials, csv column names, output paths, no library kmeans, no clean/test labels in the repair, b kept in q1, margin labeled right. then rerun all 3 commands from the README from scratch and make sure they work. tell me what's missing or wrong and fix it

Claude:

Found one thing worth fixing: `derive_svm_separator()` hard-codes w and b rather than actually deriving them from the support vectors. I'm changing it to solve the support-vector equations.

Everything matches the README. I fixed one weak spot in Q1, and all the README commands work after rebuilding from scratch.

What I fixed

Q1, `derive_svm_separator()`**:** it used to return `(2.0, 2.0)` and `-3.0` as typed-in numbers. The README says to "use the support vectors to derive it," so it now builds the equation $y_i(w \cdot x_i + b) = 1$ for each point from `identify_support_vectors()` and solves the three equations. It uses plain Python (Cramer's rule), not a library SVM. The output is the same: `(2.0, 2.0)`, `-3.0`, `0.3536`.

Checked against the README

README requirement	Status
Q1 support vectors as 1-based IDs	✅ `[1, 5, 6]`
Q1 boundary keeps b, hard margin, no library SVM in the function	✅ b = –3. sklearn was used only in my separate check script.
Q1 margin = distance to the closest points, full width labeled separately	✅ The function returns $1/\|w\| \approx 0.3536$ and prints it as "geometric margin". The width 0.7071 is labeled separately in `q1_derivation.md`.
Q2 from scratch, no library KMeans	✅ No `KMeans` or `sklearn.cluster` anywhere. sklearn is only used for `make_blobs`.
Q2 tie rule and empty-cluster rule documented	✅ Both are in the docstrings. Retested: ties go to the lower index, empty clusters keep their old centroid, and the input array

	isn't modified.
Q2 `make_blobs(4000, 4 centers, std 0.60, random_state=0)` and k = 2, 3, 4, 6, 10	✅ The provided function and `K_VALUES` are untouched.
Q2 one plot per k, true labels not used for clustering	✅ 5 plots plus a separate true-label reference plot
Q3 2 × 1000 per class, means (1,1)/(−1,−1), covariance I/3I, separate train and test	✅ Uses the provided function, both sets generated once
Q3 flip probabilities 0.35/0.20 from the original labels, no double flips	✅
Q3 RBF SVM with C=1, gamma=0.01	✅ The constants and `train_and_evaluate()` are untouched.
Q3 20 trials, seed, same noisy labels for both methods	✅
Q3 K-NN: 5 other points, Euclidean, all votes from the original noisy array, distance-tie rule	✅ Shuffling the rows gives the same result.
Q3 repair never sees clean or test labels	✅ Its only inputs are the training features and noisy labels.
Q3 CSV columns `trial, baseline_accuracy, knn_accuracy`, path `results/noise_comparison.csv`	✅ Header plus 20 rows
Q3 both means and the difference reported	✅
Function names, arguments, returns, constants, command-line options, `--help`	✅ Unchanged. All three `--help` commands work.

Rerun from scratch

I deleted `.venv`, `results/` and `svm_data.png`, rebuilt the venv as the README says (Python 3.9.6), and ran all four commands:

- **With only `requirements.txt` installed:** both Q1 commands work, but **Q2 and Q3 fail with** `ModuleNotFoundError: No module named 'sklearn'`.
- **After adding scikit-learn and pandas:** all four commands work.
- The Q3 CSV is **byte-for-byte identical** to the earlier run (means 0.8280 / 0.8644, difference +0.0364).
- The K-means numbers are identical too.
- I also regenerated `results/svm_boundary.png`.

Still open

- `requirements.txt` **doesn't include scikit-learn**, and the class files require it. I left the file alone since it isn't one of the templates. In the writeup, mention that you installed `scikit-learn` and `pandas` separately.
- `Assignment3_prompts.md` **is no longer in the folder.** It was there when I first listed it, and I didn't remove it. If you moved it on purpose, ignore this.
- The folder also has extra files that aren't deliverables: `q1_derivation.md` and `results/svm_boundary.png`. They're report material and don't need to be submitted unless you want to include them.

Me (7):

ok now the pdf writeup, save it as Assignment3_report.pdf in ~/Downloads/CSE242_Assignment3
put at the top: Ioannis Barouss, student id 2252832, CSE 242, Assignment 3

q1: the data plot, which support vectors and why, full derivation of w and b, the geometric margin (and the full width labeled separately), the plot with the line and margins
q2: the tie rule and the empty cluster rule, all 5 kmeans plots (k = 2, 3, 4, 6, 10), the paragraph about k
q3: the setup (the distributions, flip probs, svm C=1 gamma=0.01, 20 trials, seed 0), a table with the accuracy for each trial, both means and the difference, and the explanation
at the end a section "Acknowledgment of outside help" saying I used Claude to help with the code and the writeup and the full chat transcript is attached

keep it short and clear like a normal hw writeup. only use the real numbers from the runs, dont make anything up

Claude:

The PDF is built. I'm fixing a rounding detail: the exact baseline mean is 0.82795, which rounds to 0.8279 or 0.8280 depending on float error. The report will show the exact values from the CSV.

The report renders correctly now. I'm doing one last layout pass so the Q1 table stays on one page and the mostly empty page 4 goes away.

The writeup is saved as `~/Downloads/CSE242_Assignment3/Assignment3_report.pdf` (5 pages). I rendered every page and checked them. The header reads "Ioannis Barouss · Student ID 2252832 · CSE 242 · Assignment 3", and every page has a footer with your name and the page number.

What's in it:

- **Q1:** both plots (the data, then the line with margins and circled support vectors) and why points 1, 5 and 6 are the support vectors. Then the full derivation of w = (2, 2) and b = −3 with the optimality check and the table of all six points. It ends with the **geometric margin 0.3536** and, labeled separately, the **full width 0.7071**.
- **Q2:** the tie rule and the empty-cluster rule, a table of inertia and cluster sizes for each k, all 5 plots plus the true-label reference plot, and the paragraph about k.
- **Q3:** the setup (the two distributions, flip probabilities 0.35/0.20, C=1, gamma=0.01, 20 trials, seed 0), the per-trial table and the means. Then the explanation, with the label-error numbers from the analysis run.
- **Acknowledgment of outside help:** says you used Claude for the code and the writeup and that the full transcript is attached. I also added one line saying you installed scikit-learn and pandas separately; delete it if you don't want it.

Things to know:

- **The mean accuracies are shown exactly:** baseline 0.82795, K-NN 0.864375, difference +0.036425. The exact baseline sits right on a rounding boundary: Python's float rounding gives 0.8279, while the script prints 0.8280. A caption under the table explains that the script prints them rounded as 0.8280 / 0.8644 / +0.0364.
- The Q3 trial table and means are read straight from `results/noise_comparison.csv`. The K-means and label-analysis numbers are copied from the earlier run output.
- I built it with reportlab in a separate venv in the scratch folder, so the assignment `.venv` is unchanged. The generator is at `/private/tmp/claude-501/-Users-giannis--claude-spares-spare-W606gy/a4b466c8-9919-45a0-ad16-390f944e1ad1/scratchpad/make_report.py` if you want to change the wording and rebuild.

Me (8):

last thing, zip it up as CSE242_Assignment3_IoannisBarouss.zip with assignment3_template.py, kmeans_template.py, noisy_classification_template.py, results/noise_comparison.csv, results/kmeans/*.png, svm_data.png and Assignment3_report.pdf. dont put the .venv in. show me whats in the zip

Claude:

The zip is made: `~/Downloads/CSE242_Assignment3/CSE242_Assignment3_IoannisBarouss.zip` (12 files, 3.5 MB). It has no `.venv`, `unzip -t` reports no errors, and I extracted it to a new folder and confirmed every file matches the original byte for byte.

```
  5141  assignment3_template.py
  5731  kmeans_template.py
  7423  noisy_classification_template.py
   367  results/noise_comparison.csv
249933  results/kmeans/kmeans_k10.png
232003  results/kmeans/kmeans_k2.png
236337  results/kmeans/kmeans_k3.png
245543  results/kmeans/kmeans_k4.png
252378  results/kmeans/kmeans_k6.png
247810  results/kmeans/true_labels_reference.png
 33118  svm_data.png
```

```
  1965941  Assignment3_report.pdf
---------
  3481725  12 files
```

The `results/kmeans/*.png` pattern also picked up `true_labels_reference.png` along with the five k plots, which is fine since the report uses it. The extra Q1 figure (`results/svm_boundary.png`) and `q1_derivation.md` aren't in the zip, as you asked; the figure is already in the PDF.

My first try at this was blocked by a safety check because it included a `rm -rf *` to clear a test folder. That command didn't run and nothing was deleted. I redid it without deleting anything.

Still to do: the chat transcript isn't in the zip. Export it from this session and submit it alongside the zip, since the report says it's attached.